

LAB REPORT 08

Two-Dimensional Arrays in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

The goal of this lab was to build a solid understanding of two-dimensional arrays in C++ through practical programming. The session covered:

- Understanding the concept and structure of a two-dimensional array
- Declaring and initialising 2D arrays using row-column notation
- Accessing and displaying elements using nested loops
- Performing row sums, column sums, and finding the maximum value

2 Theoretical Background

What is a Two-Dimensional Array?

A **two-dimensional array** is the simplest multidimensional array in C++, best visualised as a table of rows and columns. Every element is identified by *two* indices: a row index and a column index. Internally the elements are stored in *row-major order* — each row laid out directly after the previous one in memory — which affects how efficiently elements are accessed.

Declaration and Initialisation

A 2D array is declared by specifying the data type, name, and sizes for both dimensions — e.g. `int a[3][4];`. Values can be provided using nested curly braces (each inner set representing one row), or as a flat list that fills the array row by row automatically.

Accessing Elements with Nested Loops

Each element is accessed with two subscripts: row index first, then column index. Nested for loops are the standard traversal method — the outer loop steps through rows while the inner loop steps through columns. The same pattern is used for reading values and for all array-wide computations.

3 Software Used

IDE

DEV-C++ IDE

COMPILER

C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks**Task 1 — 2D Array Declaration, Initialisation, and Display**

A 3x3 integer array named matrix is initialised with values 1–9. Nested loops walk every row and column, printing elements in a tab-separated grid.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int matrix[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) cout << matrix[i][j] << "\t";
        cout << endl;
    }
    return 0;
}
```

Task 2 — Row Sums, Column Sums, and Total Sum

A 3x3 array with fixed values. The program calculates the sum of each row, each column, and a running grand total across all elements.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int a[3][3] = {{1,3,5},{2,4,6},{7,8,9}}; int total = 0;
    for (int i = 0; i < 3; i++) {
        int rs = 0;
        for (int j = 0; j < 3; j++) rs += a[i][j];
        cout << "Row " << i << " = " << rs << endl; total += rs;
    }
    for (int j = 0; j < 3; j++) {
        int cs = 0;
        for (int i = 0; i < 3; i++) cs += a[i][j];
        cout << "Col " << j << " = " << cs << endl;
    }
    cout << "Total = " << total << endl; return 0;
}
```

Task 3 — Finding the Maximum Value in a 2D Array

A 4x4 array is scanned with nested loops. Starting from the first element as the assumed maximum, any larger value found replaces it. The final maximum is printed after the full scan.

● ● ● C++ Code

```
#include <iostream>
using namespace std;
int main() {
    int grid[4][4] = {{3,17,8,2},{45,6,23,11},{9,31,4,56},{12,7,38,19}};
    int maxVal = grid[0][0];
    for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++)
            if (grid[i][j] > maxVal) maxVal = grid[i][j];
    cout << "Maximum = " << maxVal << endl; return 0;
}
```

5 Conclusion

This lab provided practical exposure to 2D arrays in C++, covering declaration, initialisation, element access, and computation. The tasks demonstrated how nested loops and 2D arrays work together for row/column operations and value searches. These techniques are foundational for matrix operations and table-driven algorithms.